

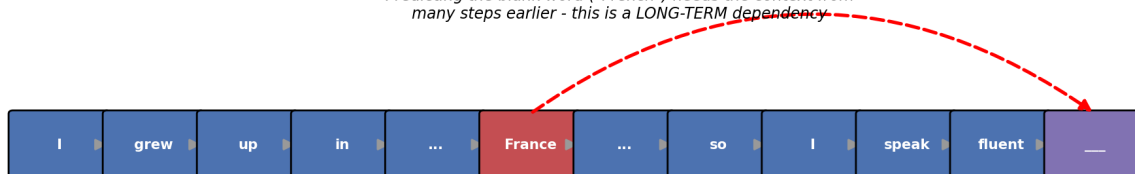
Problems with RNN

The Long-Term Dependency Problem & Its Solutions

A Visual Guide to RNN Limitations, Vanishing/Exploding Gradients, and How LSTM, GRU & Modern Fixes Solve Them

The Long-Term Dependency Problem

Predicting the blank word ("French") needs the context from many steps earlier - this is a LONG-TERM dependency



Covers: Key problems with vanilla RNNs • The long-term dependency problem • Why it happens (vanishing/exploding gradients) • Gated solutions (LSTM & GRU) • Other practical fixes • Comparison summary

1. Key Problems with Vanilla RNNs

Despite solving the memory problem that feedforward networks have, the basic ("vanilla") RNN comes with its own set of serious limitations. The four biggest ones are:

- **1. Long-term dependency problem** - struggles to connect information across many time steps.
- **2. Vanishing gradients** - gradients shrink toward zero during training, so early time steps barely learn.
- **3. Exploding gradients** - gradients can also grow uncontrollably large, causing unstable training.
- **4. Sequential computation** - each step must wait for the previous one, so training can't be parallelized across time, making RNNs slow on long sequences.

RNNs Process Sequentially - Cannot Parallelize Across Time

Must wait for step t before computing step $t+1$ -> slow training on long sequences

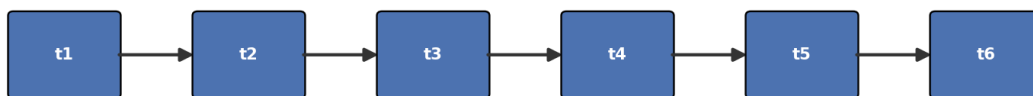


Figure 1: RNNs process one time step at a time, which limits parallel training compared to modern architectures like Transformers.

Of these, the **long-term dependency problem** is the most fundamental and famous limitation - it's the reason LSTM and GRU were invented in the first place. The rest of this guide focuses on it in detail.

2. What Is the Long-Term Dependency Problem?

In theory, an RNN's hidden state can carry information indefinitely far into the future. In practice, it can't. As the gap between a relevant piece of information and the point where it's needed grows, a vanilla RNN becomes increasingly unable to connect the two. This is called the **long-term (or long-range) dependency problem**.

The Long-Term Dependency Problem

Predicting the blank word ("French") needs the context from many steps earlier - this is a LONG-TERM dependency.

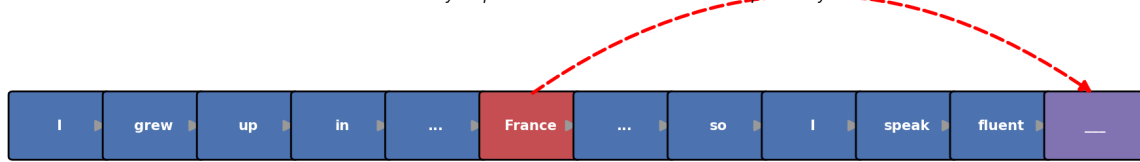


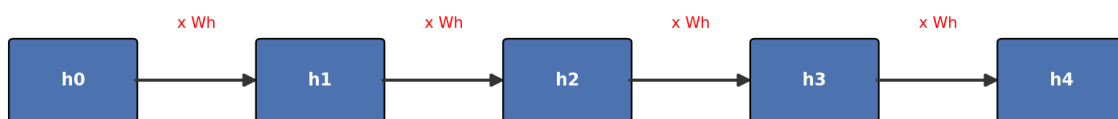
Figure 2: To correctly predict 'French', the network must remember 'France' from many words earlier - a long-term dependency.

For short gaps (e.g. "the sky is ____" -> "blue"), a vanilla RNN does fine, since the relevant context appeared only a step or two earlier. But for long gaps - a name mentioned paragraphs ago, a subject far before its verb, a plot detail from early in a story - the signal from that information gets weaker and weaker as it's carried forward, until it effectively disappears.

3. Why It Happens: Vanishing & Exploding Gradients

The root cause is mathematical. During training, we use **Backpropagation Through Time (BPTT)** to compute how much each weight contributed to the final error. Because the RNN reuses the same hidden-to-hidden weight matrix W_h at every time step, computing the gradient for an early time step requires multiplying by W_h (and the derivative of the activation function) once for every step in between.

Why Gradients Vanish: Repeated Multiplication via the Chain Rule



$$dL/dh_0 = dL/dh_4 * W_h * W_h * W_h * W_h \quad (\text{if } |W_h| < 1, \text{ product shrinks toward 0 fast})$$

Figure 3: The chain rule forces repeated multiplication by W_h across every time step in between.

This repeated multiplication behaves like compound interest, but in reverse:

- If the multiplied terms are consistently **less than 1**, the product shrinks exponentially toward zero as the number of steps grows -> **vanishing gradients**. Early time steps stop receiving any meaningful learning signal.

- If the multiplied terms are consistently **greater than 1**, the product grows exponentially -> **exploding gradients**. Weight updates become huge and unstable, and training can literally produce NaN values.

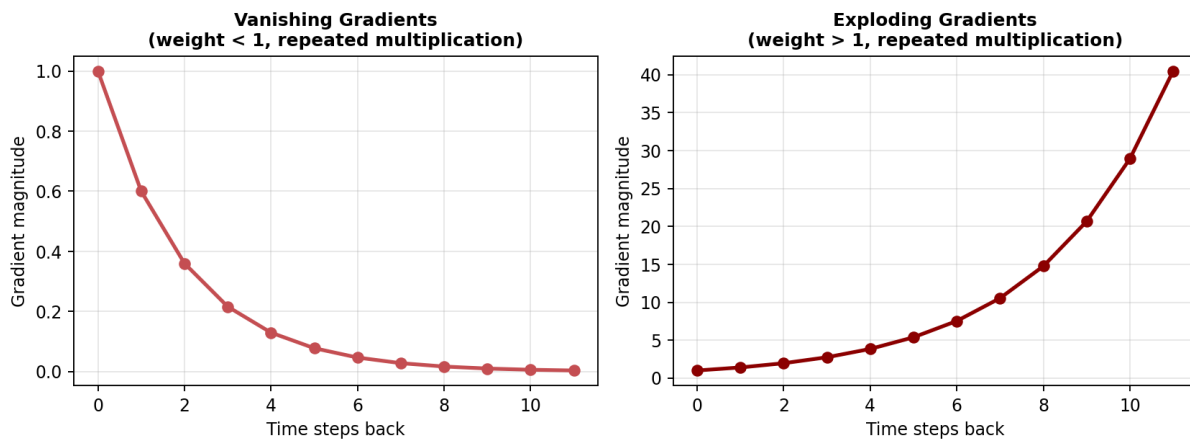


Figure 4: Both failure modes come from the same repeated-multiplication mechanism, just in opposite directions.

The tanh and sigmoid activations traditionally used in RNNs make vanishing gradients especially likely, since their derivatives are always less than or equal to 0.25-1.0, further shrinking the gradient at every step.

4. Solutions to the Long-Term Dependency Problem

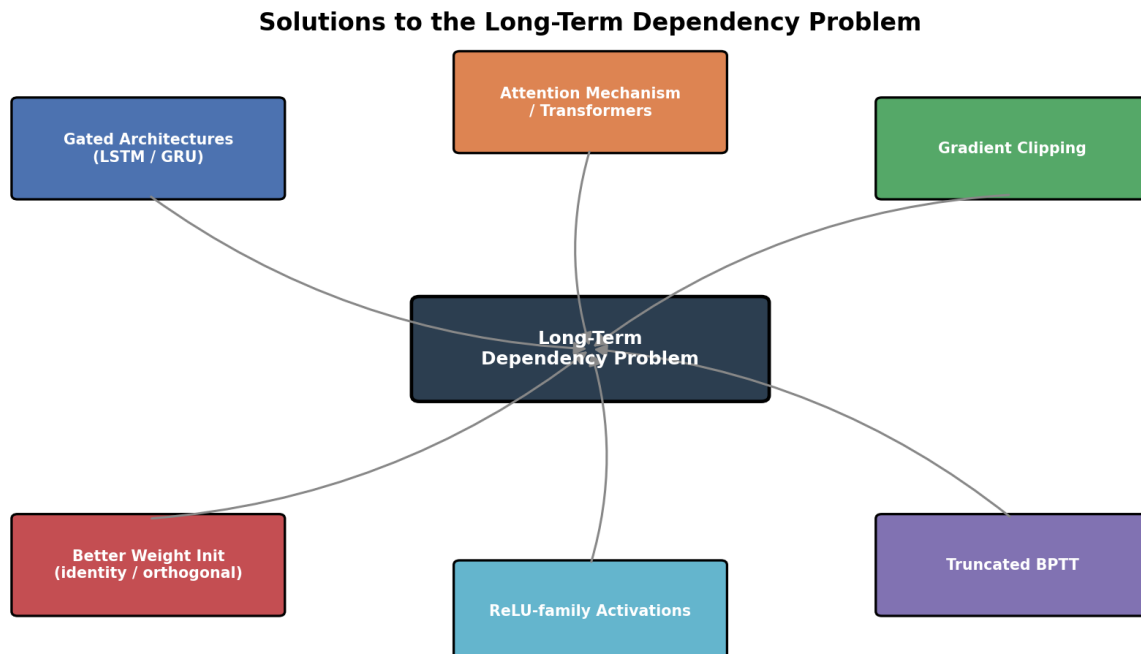


Figure 5: The main families of solutions developed to address vanishing/exploding gradients in RNNs.

4.1 Gated Architectures: LSTM

The **Long Short-Term Memory (LSTM)** network is the classic fix. It introduces a separate **cell state** that runs through the whole sequence with only minor, controlled changes at each step, plus three **gates** that regulate what information flows through:

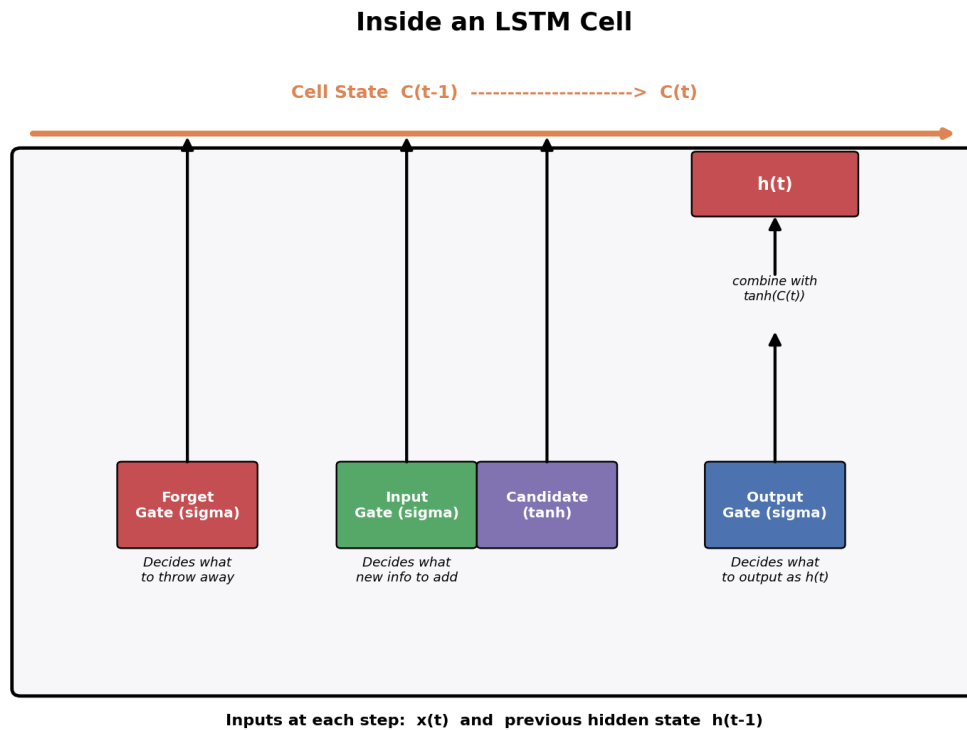


Figure 6: The LSTM cell - the cell state (top) acts like a conveyor belt that can carry information across many time steps largely unchanged.

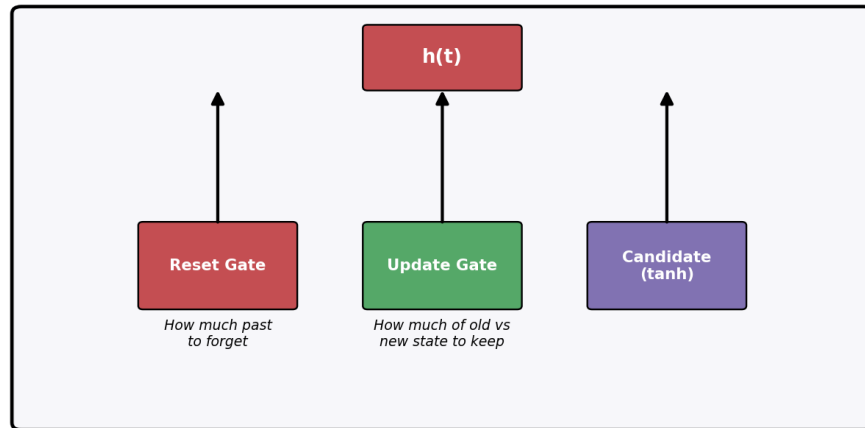
- **Forget gate** - decides what to discard from the old cell state.
- **Input gate** - decides what new information to add to the cell state.
- **Output gate** - decides what part of the cell state becomes the visible hidden state / output.

Because the cell state is updated with addition (rather than repeated multiplication through a single weight matrix), gradients can flow backward across many time steps with far less decay.

4.2 Gated Architectures: GRU

The **Gated Recurrent Unit (GRU)** is a simplified alternative to the LSTM. It merges the cell state and hidden state into one, and uses only two gates instead of four internal gate computations, making it faster to train while achieving comparable performance on many tasks.

Inside a GRU Cell (Simplified vs LSTM)



Only 2 gates (vs 4 in LSTM) -> fewer parameters, faster to train, similar performance

Figure 7: The GRU cell - a leaner design with a Reset gate and an Update gate.

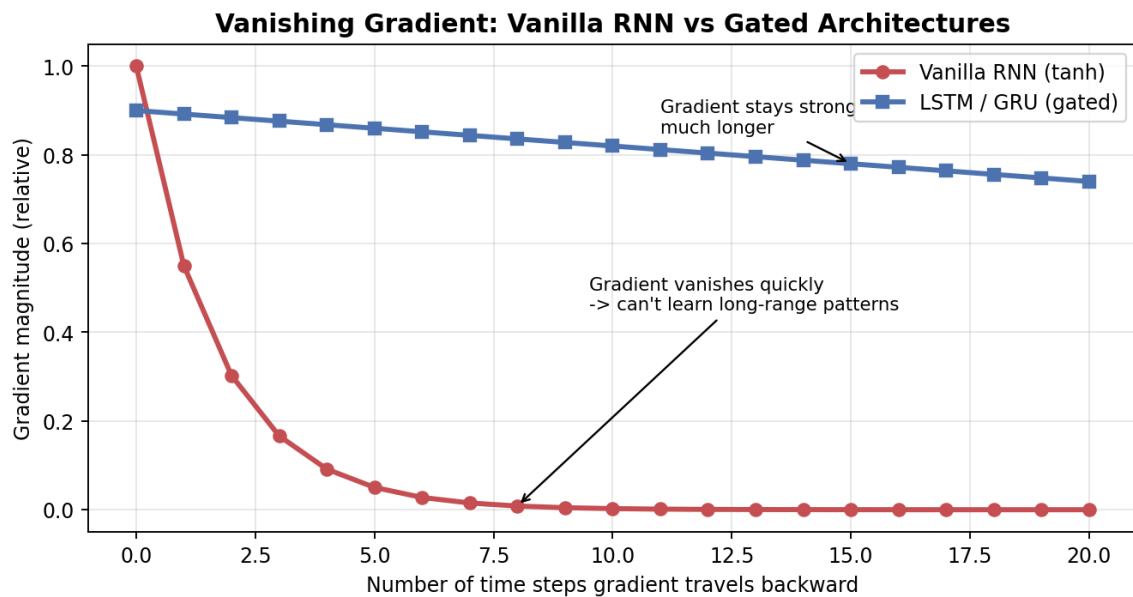


Figure 8: Gated architectures (LSTM/GRU) preserve gradient magnitude far longer than a vanilla RNN, enabling learning of long-range dependencies.

4.3 Other Practical Fixes

- **Gradient clipping** - cap the gradient's norm at a maximum threshold before the weight update, directly preventing exploding gradients (does not fix vanishing gradients).
- **Truncated BPTT** - backpropagate only a limited number of steps instead of the full sequence, trading some long-range learning ability for speed and stability.
- **Better weight initialization** - initializing W_h as an identity or orthogonal matrix keeps its eigenvalues near 1, reducing early vanishing/exploding tendencies.
- **ReLU-family activations** - replacing tanh/sigmoid with ReLU (careful initialization needed) avoids the extra gradient-shrinking effect of saturating activation derivatives.
- **Attention mechanisms / Transformers** - instead of relying on a single hidden state to carry information forward step by step, attention lets the model directly 'look back' at any earlier position in the sequence, sidestepping the long-chain gradient problem entirely.
- **Skip / residual connections** - shortcut connections that let gradients bypass several time steps directly, similar in spirit to residual networks (ResNets) in deep CNNs.

In practice, most modern sequence models either use LSTM/GRU cells for moderate-length dependencies, or move to attention-based Transformer architectures entirely when very long-range context is essential (e.g. long documents, whole conversations).

5. Summary Table

Problem	Cause	Main Solution(s)
Long-term dependency	Signal from early steps decays as it's carried forward	LSTM, GRU, Attention/Transformers
Vanishing gradients	Repeated multiplication by $W_h < 1$ and small activation derivatives	Gradient clipping, ReLU-family activations, good initialization
Exploding gradients	Repeated multiplication by $W_h > 1$	Gradient clipping, good initialization
Slow / sequential training	Each step depends on the previous one	Truncated BPTT, Transformer-style parallel attention

Bottom line: vanilla RNNs are conceptually elegant but structurally limited in how far back they can 'remember.' LSTM and GRU fixed most of the vanishing-gradient issue with gating, and modern attention-based models go a step further by letting a network access any past position directly - which is a big part of why Transformers have largely replaced RNNs for tasks needing very long-range context.